

Program-1: Write a program to demonstrate the poly alphabetic Cipher

Code:

```
# PROGRAM 1: Poly-Alphabetic Cipher (Vigenère Cipher)
```

```
# Input from file → Output to file
```

```
def vigenere_encrypt(plaintext, key):
```

```
    key = key.upper()
```

```
    result = ""
```

```
    key_index = 0
```

```
    for ch in plaintext:
```

```
        if ch.isalpha():
```

```
            shift = ord(key[key_index % len(key)]) - ord('A')
```

```
            base = 'A' if ch.isupper() else 'a'
```

```
            encrypted = chr((ord(ch) - ord(base) + shift) % 26 + ord(base))
```

```
            result += encrypted
```

```
            key_index += 1
```

```
        else:
```

```
            result += ch
```

```
    return result
```

```
input_file = "input.txt"
```

```
output_file = "polyalph_output.txt"
```

```
key = input("Enter key for Poly-Alphabetic Cipher: ")
```

```
with open(input_file, "r") as f:
    plaintext = f.read()

ciphertext = vigenere_encrypt(plaintext, key)

with open(output_file, "w") as f:
    f.write(ciphertext)

print("Poly-Alphabetic Cipher Encryption Completed!")
print(f"Ciphertext written to {output_file}")
```

Output:

Enter key for Poly-Alphabetic Cipher: Encrypt
Poly-Alphabetic Cipher Encryption Completed!
Ciphertext written to polyalph_output.txt

Plain Text: my name is meet shah

Cipher Text output: ql prkt bw zgvr haeu

Program-2: Write a program to demonstrate the playfair cipher

Code:

```
# PROGRAM 2: Playfair Cipher Encryption
```

```
# Input from file → Output to file
```

```
def generate_matrix(key):
    key = key.lower().replace("j", "i")
    matrix = []
    seen = set()
    for ch in key:
        if ch.isalpha() and ch not in seen:
            matrix.append(ch)
            seen.add(ch)
    for ch in "abcdefghijklmnopqrstuvwxyz": # no j
        if ch not in seen:
            matrix.append(ch)
            seen.add(ch)

    # Convert to 5x5 matrix
    return [matrix[i:i+5] for i in range(0, 25, 5)]

def find_position(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j
```

```
return None
```

```
def playfair_prepare(plaintext):  
    plaintext = plaintext.lower().replace(" ", "").replace("j", "i")  
    result = ""  
    i = 0  
    while i < len(plaintext):  
        a = plaintext[i]  
        b = ""  
        if i + 1 < len(plaintext):  
            b = plaintext[i + 1]  
        else:  
            b = "x"  
        if a == b:  
            result += a + "x"  
            i += 1  
        else:  
            result += a + b  
            i += 2  
    if len(result) % 2 == 1: # pad last  
        result += "x"  
    return result  
  
def playfair_encrypt(plaintext, matrix):  
    ciphertext = ""  
    plaintext = playfair_prepare(plaintext)  
    for i in range(0, len(plaintext), 2):
```

```

a = plaintext[i]
b = plaintext[i + 1]
r1, c1 = find_position(matrix, a)
r2, c2 = find_position(matrix, b)

# Case 1: Same row
if r1 == r2:
    ciphertext += matrix[r1][(c1 + 1) % 5]
    ciphertext += matrix[r2][(c2 + 1) % 5]

# Case 2: Same column
elif c1 == c2:
    ciphertext += matrix[(r1 + 1) % 5][c1]
    ciphertext += matrix[(r2 + 1) % 5][c2]

# Case 3: Rectangle
else:
    ciphertext += matrix[r1][c2]
    ciphertext += matrix[r2][c1]
return ciphertext

# -----
# FILE I/O + Execution
# -----

input_file = "input.txt"

```

```
output_file = "playfair_output.txt"
key = input("Enter Playfair Cipher Key: ")
with open(input_file, "r") as f:
    plaintext = f.read()
matrix = generate_matrix(key)
ciphertext = playfair_encrypt(plaintext, matrix)
with open(output_file, "w") as f:
    f.write(ciphertext)
print("Playfair Cipher Encryption Completed!")
print(f"Ciphertext written to {output_file}")
```

Output:

Enter Playfair Cipher Key: Encrypt

Playfair Cipher Encryption Completed!

Ciphertext written to playfair_output.txt

Plain Text: my name is meet shah

Cipher Text output: snctlnkqlnnpokho